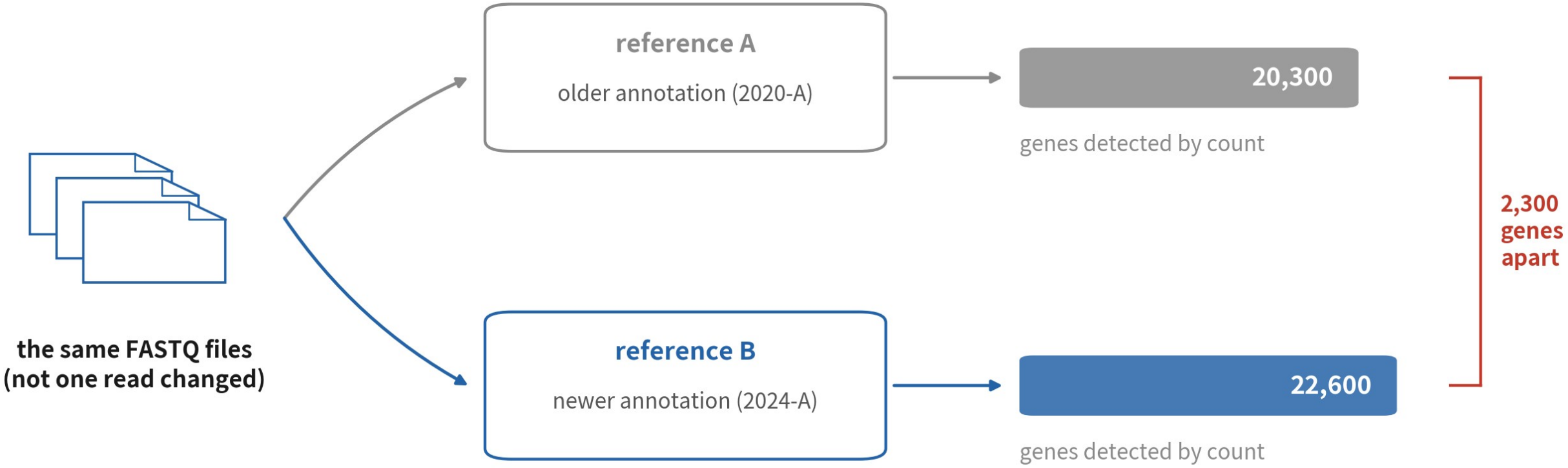
B2 Series B

Reference genomes and FASTQ: mkref and mkfastq

Know what each FASTQ file holds, where the reference comes from, and how to build your own.

About 35 min | Prerequisites: B1 | Data: PBMC 1k FASTQ (10x)

Same FASTQ, two references



illustrative numbers **same reads, same program — the whole gap lives in the reference. where?**

Not one read changed, yet the gene counts differ by over 2,000 — where is the gap?

WHERE THIS EPISODE STARTS

Half of your result is decided before count even starts.

Wrong FASTQ or wrong reference, and no amount of beautiful downstream work can save you.

What you will learn

1. State what R1 / R2 / I1 each hold, and why their lengths are what they are

2. Decide when the 10x prebuilt reference suffices and when you must build with mkref

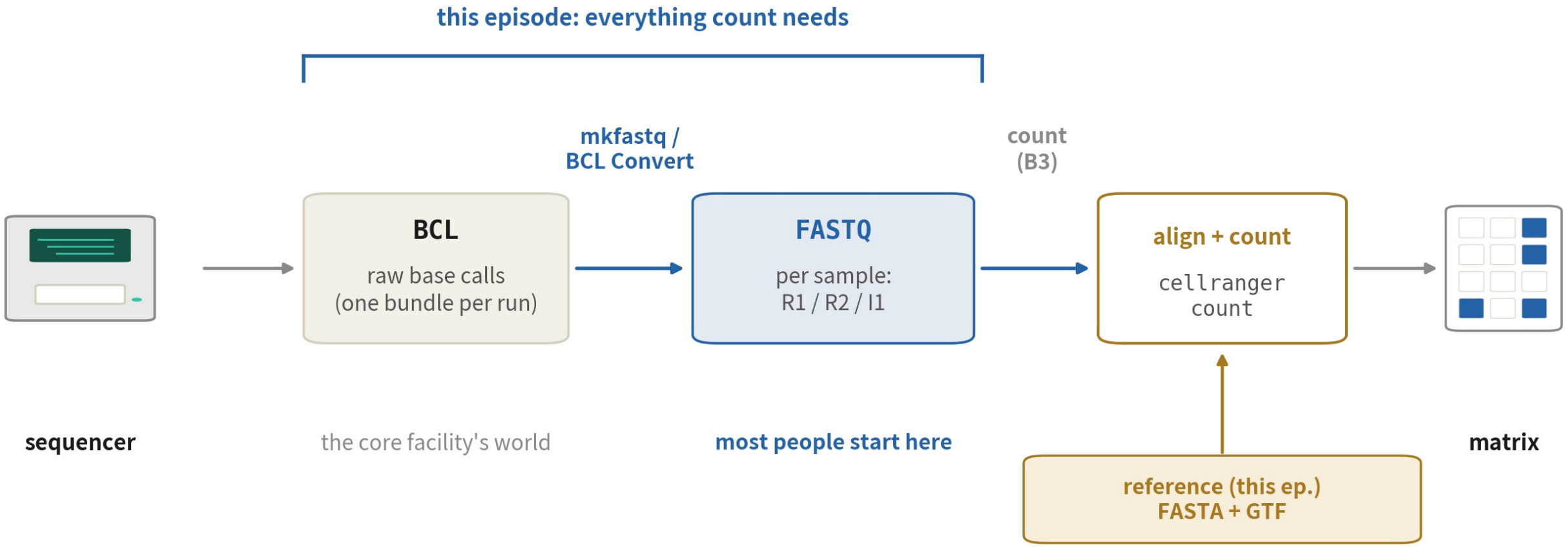
3. Explain how the GTF and its filtering decide which genes ever get counted

01

From sequencer to matrix

Map the data flow first — then see which steps are yours

The data flow before count



BCL → FASTQ → (align + count) → matrix; the reference feeds in from the side

A reference is two things: sequence + annotation

genome.fa (FASTA) — the map

```
>chr1
AGCTTACCGTTAGCCTAAGGCTTACGATCC...
```

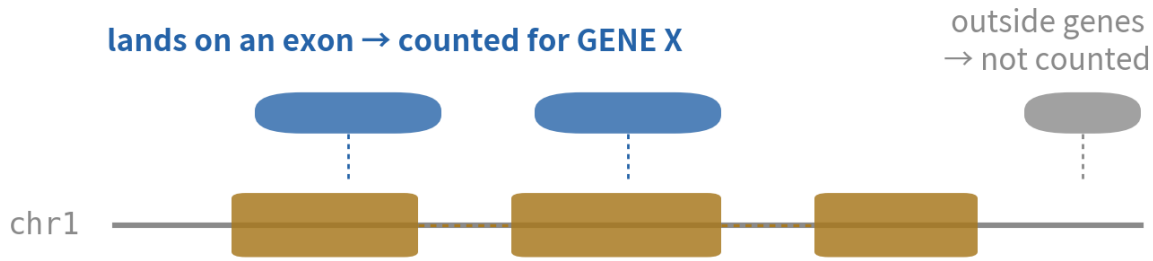
the full sequence of every chromosome

genes.gtf (GTF) — the landmarks

```
chr1  gene  11869  14409  DDX11L1
chr1  exon  11869  12227  ...
```

which coordinates belong to which gene

the two files together turn a read into a count



GTF: these exons together are GENE X

FASTA decides where a read can align;
GTF decides whether that spot counts, and for which gene

versions must match: same genome build, same coordinates

- FASTA: the genome — where reads can align
- GTF: the annotation — whether that spot counts, and for whom
- the two must come from matching releases
- the cold-open gap hides in the GTF half

BEFORE YOU PANIC

Most of this pipeline, most people never run themselves.

Cores hand you FASTQ; human and mouse have prebuilt references. But you must be able to read every file — when things break, those who can't are stuck re-sequencing.

02

Install Cell Ranger

Linux only, 64 GB of RAM, registration required

System requirements: is your machine even eligible

operating system	CPU	memory	disk
Linux x86-64 only	**8 cores minimum**	**64 GB minimum**	**hundreds of GB**
no macOS, no Windows; run it on a server	16 recommended; more cores, faster runs	a hard floor for human data; too little kills the job	BCL, FASTQ and outputs are all huge

even the download is gated: register at 10x first to get a (time-limited) link

- Linux x86-64 only — borrow a server if needed
- 64 GB of RAM is a hard floor for human data
- the download link requires registering with 10x

Download, unpack, add to PATH

```
# 10x 官網註冊後，網站會給你一段帶簽名的 curl 指令，長這樣：
curl -o cellranger-9.0.1.tar.gz "https://cf.10xgenomics.com/..."

tar -xzf cellranger-9.0.1.tar.gz
export PATH=$PWD/cellranger-9.0.1:$PATH   # 寫進 ~/.bashrc 才常駐

cellranger --version
```

```
cellranger cellranger-9.0.1
```

Download links expire — regenerate on the site; your version number may differ

Verify the install: testrun

```
cellranger testrun --id=check_install
#  用內建的迷你資料集跑一次完整流程，約 5–10 分鐘
```

```
Martian Runtime - v4.x
Running preflight checks (please wait)...
...
Pipestance completed successfully!
```

The line you care about is the last one: Pipestance completed successfully

03

Anatomy of a FASTQ

Three files, three questions: who, what, which sample

One read, exactly four lines

```
@A00228:279:HFWFVDMXX:1:1101:8828:1000 1:N:0:ACGTACGT

CTACATTCAGCCACCAGGTGTGAGTTACGCAT

+

FFFFFFFF:FFFFFFFFFFFF,FFFFFFFFF
```

— line 1: read ID — machine, run, flowcell, tile

— line 2: the sequence itself (A/C/G/T/N)

— line 3: separator — just a plus sign

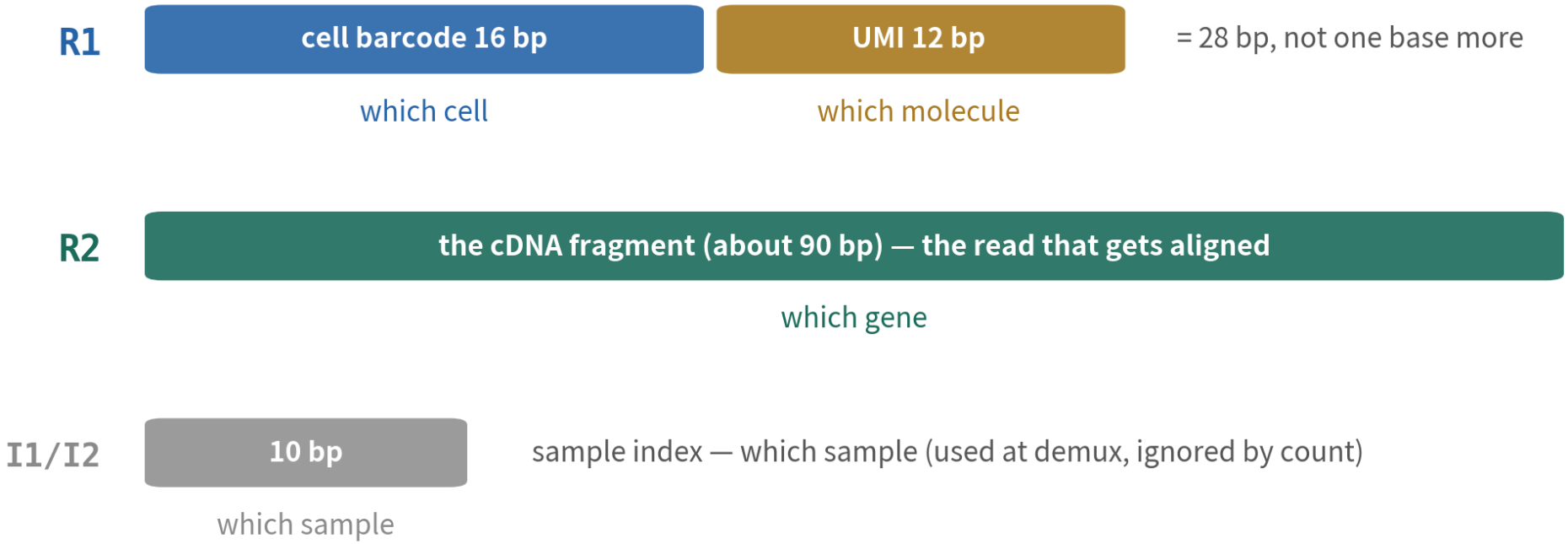
— line 4: quality — one character per base above

one read = exactly four lines; a FASTQ file is tens of millions of these

A FASTQ is just a compressed text file of tens of millions of four-line records

What R1, R2 and I1 each hold

three files, three questions: who (R1), what (R2), which sample (I1/I2)



R1 length is the chemistry's ID card: 28 bp = v3/v4 (16+12), 26 bp = v2 (16+10)

This is the 10x convention — completely different from bulk R1/R2. The biggest trap of the episode

Peek inside R1

```
zcat pbmc_1k_v3_S1_L001_R1_001.fastq.gz | head -4
```

```
@A00228:279:HFWFVDMXX:1:1101:8828:1000 1:N:0:ACGTACGT
CTACATTCAGCCACCAGGTGTGAGTTAC
+
FFFFFFFF:FFFFFFFFFFFF,FFFFF
```

Line 2 is exactly 28 letters: 16 barcode + 12 UMI — this is not the read that gets aligned

Check read lengths: one awk line

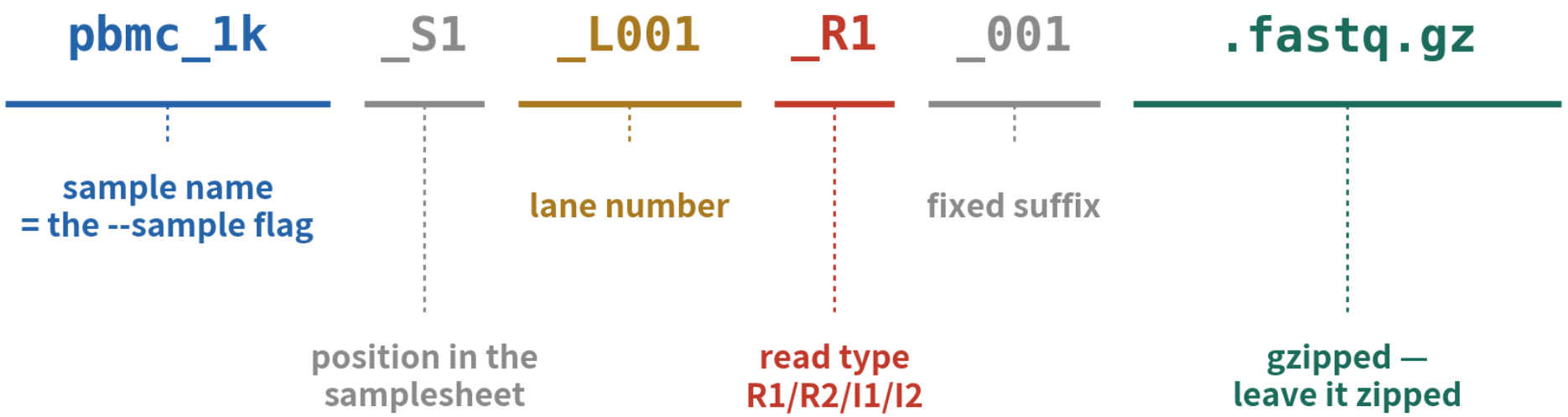
```
# R2 才是 cDNA；順便驗證 R1 的讀長是不是 28
zcat pbmc_1k_v3_S1_L001_R1_001.fastq.gz | head -4000 |
  awk 'NR % 4 == 2 { print length($0) }' | sort | uniq -c
```

```
   1000 28
```

28 means v3/v4 chemistry, 26 means v2. Wrong length? Ask the core before forcing a run

The filename is part of the spec

cellranger accepts exactly this pattern (bcl2fastq naming)



one segment off — and count pretends the file does not exist

cellranger locates files by name: sample, S index, lane, read type — every segment must be right

10x R1/R2 is not bulk R1/R2

Same names, different meaning — the classic trap for bulk veterans

bulk RNA-seq

R1 and R2 are both cDNA

- paired-end: two ends of one fragment
- both files go to the aligner
- symmetric — lose one, run single-end

10x scRNA-seq

R1 is barcodes; only R2 is cDNA

- R1 = cell barcode + UMI, 28 bp
- only R2 is aligned to the genome
- asymmetric — and both are indispensable

At this point you can read a FASTQ

Three files

R1 = who (barcode+UMI), R2 = what (cDNA), I1/I2 = which sample

Two checks

zcat the read lengths; verify the filename pattern

One big trap

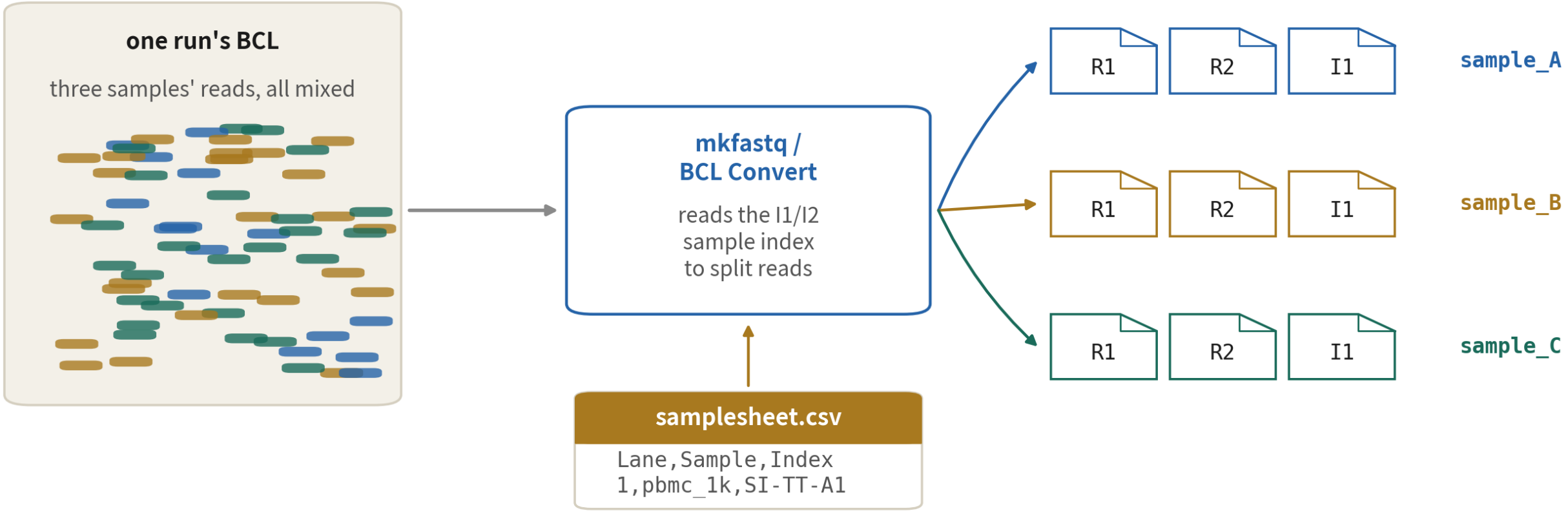
R1/R2 mean something different from bulk — never assume

04

BCL to FASTQ: mkfastq

You will rarely run it — but you must know what it did

What mkfastq does: demultiplexing



the core facility usually runs this — you receive the files on the right

It reads the sample index to split one run's mixed reads into per-sample FASTQ files

mkfastq: the command and samplesheet

```
# samplesheet.csv ——  三欄就夠: lane、樣本名、10x index 組名
# Lane,Sample,Index
# 1,pbmc_1k,SI-TT-A1

cellranger mkfastq \
  --run=/data/runs/240801_A00228_0279_BHFWFVDMXX \
  --csv=samplesheet.csv \
  --output-dir=fastq_out
```

Index is a set name, not a sequence; the sample name becomes the FASTQ filename prefix

What you actually need to know about mkfastq

It is a wrapper

under the hood it is Illumina's bcl2fastq; 10x now steers you to BCL Convert

Usually not your job

cores usually deliver demultiplexed FASTQ — start from the next step

But you must inspect

read lengths, filenames, file sizes — run the previous section's checks

05

The reference: prebuilt or custom

Nine in ten just download; the rest need mkgtf + mkref

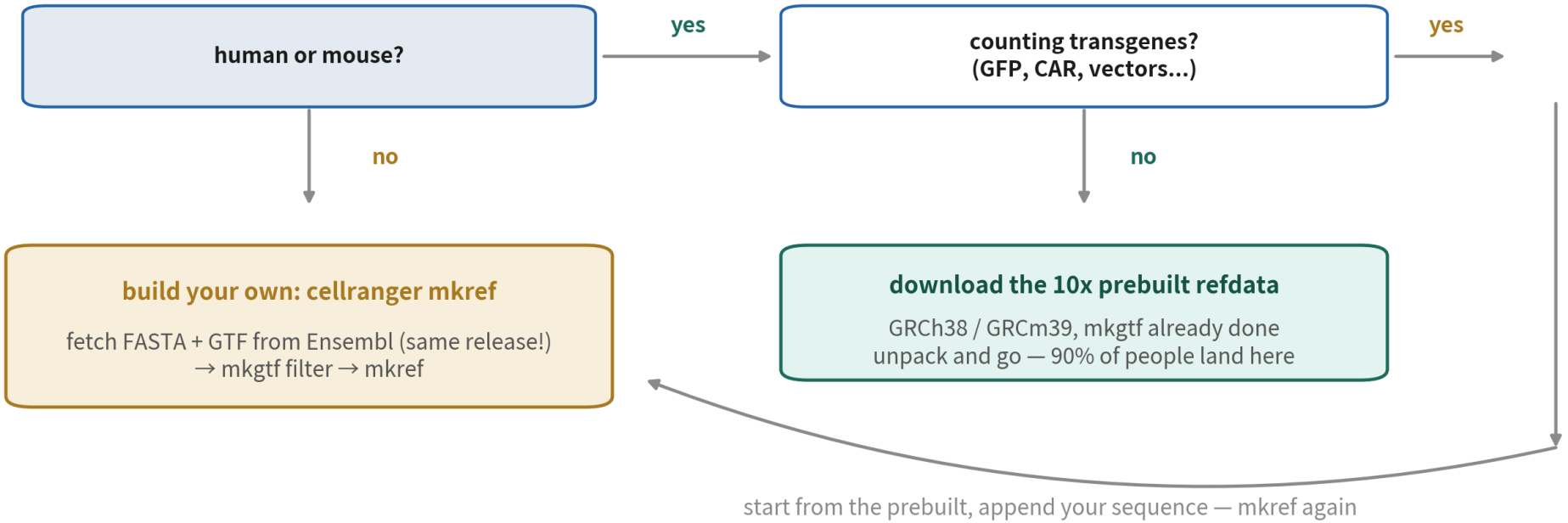
The prebuilt reference: download and go

```
# 人類 GRCh38（2024-A，GENCODE v44 注釋），約 11 GB
curl -O https://cf.10xgenomics.com/supp/cell-exp/\
refdata-gex-GRCh38-2024-A.tar.gz
tar -xzf refdata-gex-GRCh38-2024-A.tar.gz
ls refdata-gex-GRCh38-2024-A
```

```
fasta/          genome.fa 基因組序列
genes/          genes.gtf.gz 注釋（已過濾）
star/           STAR 比對索引
reference.json  版本與來源紀錄
```

reference.json records the genome and annotation versions — copy it into your methods section

When you must build your own



two questions decide whether you ever touch mkref

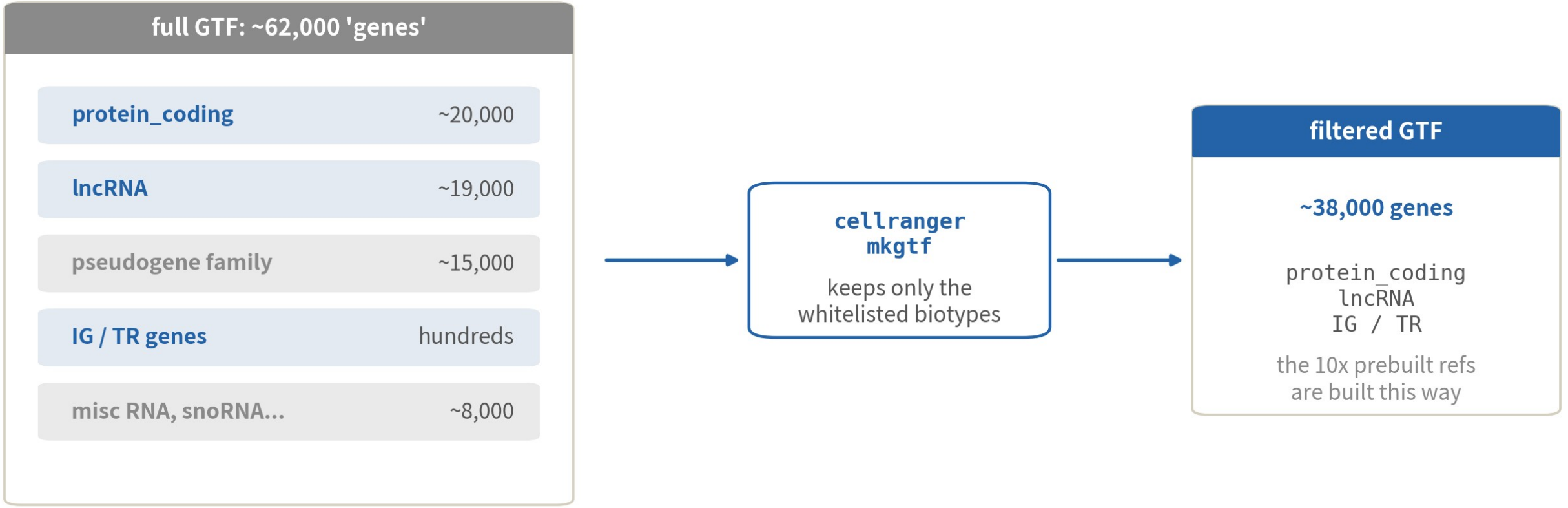
- human/mouse, no transgene → prebuilt; note the version
- non-model species → fetch matching FASTA + GTF, build
- counting GFP, CAR etc. → extend the prebuilt and rebuild

Step one: filter the GTF with mkgtf

```
cellranger mkgtf \
  Danio_rerio.GRCz11.110.gtf \
  Danio_rerio.GRCz11.110.filtered.gtf \
  --attribute=gene_biotype:protein_coding \
  --attribute=gene_biotype:lncRNA \
  --attribute=gene_biotype:IG_C_gene \
  --attribute=gene_biotype:TR_C_gene
# 以斑馬魚為例；沒列進白名單的 biotype 整類移除
```

--attribute is a whitelist; keeping protein_coding and lncRNA mirrors the 10x prebuilt recipe

What mkgtf filters away



filtered biotypes (pseudogenes etc.) cease to exist — they are noisy read sinks

Pseudogenes mimic their parents and soak up reads as multimappers — filtering protects the counts

Step two: build the index with mkref

```
cellranger mkref \
  --genome=GRCz11_cr \
  --fasta=Danio_rerio.GRCz11.dna.primary_assembly.fa \
  --genes=Danio_rerio.GRCz11.110.filtered.gtf \
  --nthreads=8 --memgb=64
# 人類基因組約 1–2 小時；輸出資料夾之後餵給 count
```

```
Creating new reference folder at ./GRCz11_cr
...building STAR index (this can take a while)...
>>> Reference successfully created! <<<
```

FASTA and GTF must share a release — mixing versions is the classic way custom builds die

Adding a transgene: touch both files

genome_plus_gfp.fa

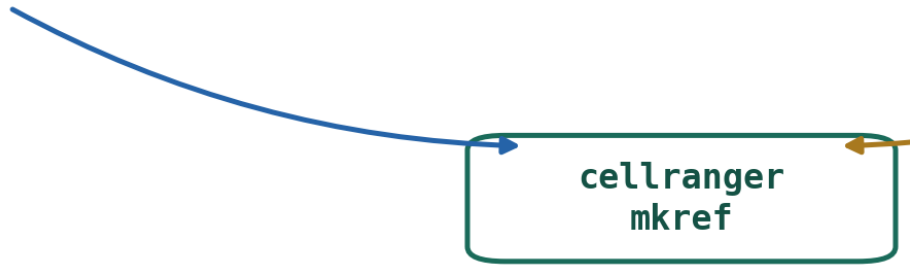
```
>chr1  AGCTTACC...
>chr2  TTGACCAT...
...
>GFP  ATGGTGAGCAAGGGCGAG...
```

genes_plus_gfp.gtf

```
chr1  gene  11869  ...
chr1  exon  11869  ...
...
GFP exon 1 720 gene_id "GFP"
```

1) append the sequence to the FASTA
(a new 720 bp 'mini-chromosome')

2) add a GFP exon line to the GTF
(tell the counter this sequence is a gene)



both edits are mandatory — FASTA alone means GFP counts stay at zero (see traps)

GFP rides along as a mini-chromosome, with a matching gene entry added to the GTF

The actual commands for adding GFP

```
cat refdata/fasta/genome.fa GFP.fa > genome_plus_gfp.fa

# GTF 加一行 exon （gene_id / transcript_id 都叫 GFP）
echo -e 'GFP\tcustom\texon\t1\t720\t.\t+\t.\t'\
'gene_id "GFP"; transcript_id "GFP"; gene_name "GFP";'\
 >> genes_plus_gfp.gtf

cellranger mkref --genome=GRCh38_gfp \
  --fasta=genome_plus_gfp.fa --genes=genes_plus_gfp.gtf
```

The full runnable version lives in R/B02_mkref_mkfastq.sh; always verify GFP is nonzero after

COLD OPEN, RESOLVED

2,000 genes apart with identical reads — the GTF changed: annotation version and filter.

This is why methods sections state the reference version. It is the first line to check when others cannot reproduce you.

06

Where people get hurt

Three traps — each with its consequences shown for real

Trap 1: feeding R1 as the cDNA read

What it is

Swapping R1/R2 out of bulk habit, or mispairing files at download.

Why it happens

In bulk, R1 is cDNA. Same name — who would expect a different meaning?

What it costs you

cellranger finds no valid barcodes in R1, the rate collapses toward zero, and the run aborts.

What a swapped run looks like

same files, just swapped

correct: R1=barcode, R2=cDNA

Valid Barcodes
97.2%

Reads Mapped to Genome
91.5%

runs clean

swapped: R2 fed into R1's slot

Valid Barcodes
0.3%

Reads Mapped to Genome
—

aborts mid-run

illustrative numbers

cellranger finds no valid barcodes in R1 — and the error message says so explicitly

The good news: this trap fails loudly — the error says the barcodes do not match the whitelist

Trap 2: nonconforming filenames — count refuses

What it is

Files renamed (sample.R1.fq.gz style), or the _001 stripped by a cloud download.

Why it happens

cellranger finds files by the bcl2fastq naming pattern — you point at a folder, not at files.

What it costs you

It reports 'No input FASTQs were found' with the files sitting right there — until you restore standard names.

Trap 2, live — and the fix

```
# [踩雷示範] 檔名少了 _S1_L001 與 _001：
# fastq_renamed/pbmc1k_R1.fq.gz pbmc1k_R2.fq.gz
cellranger count --id=demo --sample=pbmc1k \
  --fastqs=fastq_renamed --transcriptome=refdata-...

# 解法：改回 bcl2fastq 命名（樣本名要與 --sample 一致）
mv pbmc1k_R1.fq.gz pbmc1k_S1_L001_R1_001.fastq.gz
mv pbmc1k_R2.fq.gz pbmc1k_S1_L001_R2_001.fastq.gz
```

```
error: No input FASTQs were found for the requested
parameters. Please check your --fastqs and --sample.
```

Renaming touches only the name, never the data; --sample must match the prefix

Trap 3: GFP in the FASTA, forgotten in the GTF

What it is

You appended GFP to the FASTA but never added the matching GTF entry.

Why it happens

mkref will not complain — an unannotated extra sequence is perfectly legal.

What it costs you

count runs clean with zero warnings, and GFP reads 0 in every cell. Found weeks later; experiment redone.

The quiet zero, visualized

count finishes cleanly — no error anywhere

every cell reads 0 — the whole map is grey

the priciest kind of trap: no error, just a quiet zero — often found weeks later

One-line check: zcat genes.gtf.gz | grep GFP — run it after mkref, before you ever launch count

TRAPS, SUMMED UP

Loud errors are a gift.

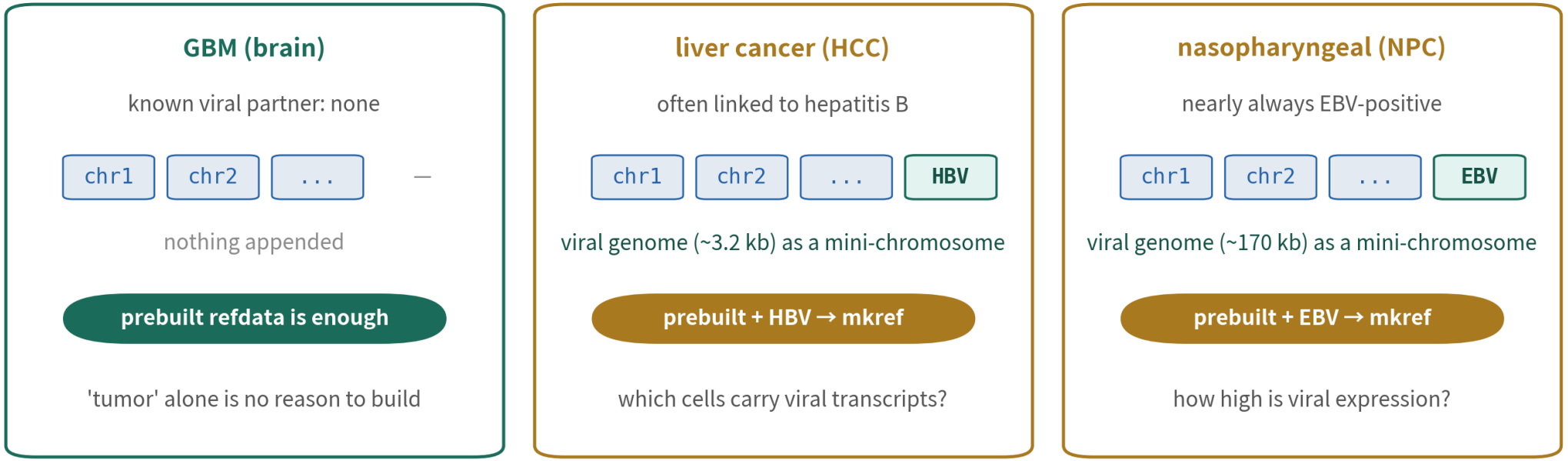
The quiet ones you must hunt yourself.

Traps 1 and 2 fail loudly — follow the message. Trap 3 never does. Run the checklist before launching, not after the damage.

Strategy Room: Complex Samples

The same reference decision — and how the answer changes on a tumor

Tumor references: make the virus countable



the recipe = this episode's GFP move: extend FASTA and GTF, rerun mkref — skip it and trap 3's quiet zero returns

- liver cancer is often HBV-linked, NPC nearly always carries EBV — no virus in the prebuilt
- the recipe is this episode's GFP move: extend FASTA and GTF, rerun mkref
- skip it and you get trap 3: viral reads quietly dropped, infection forever invisible
- GBM has no known viral partner — same word 'tumor', different answer

PDX samples: a joint human + mouse reference

a PDX sample

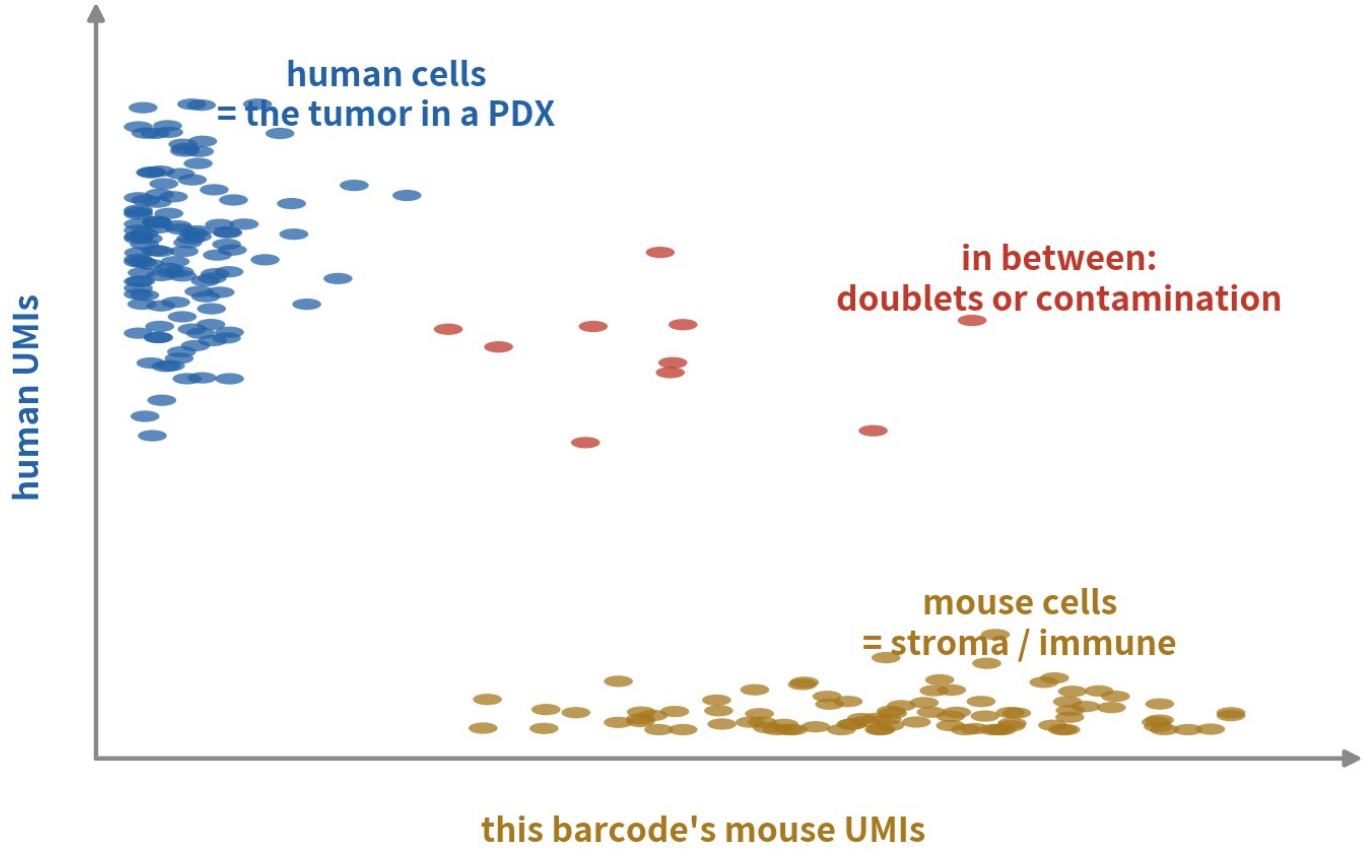
a patient's tumor, grown in a mouse

human: the malignant cells

mouse: stroma + immune cells

→ align to a joint human + mouse reference (10x ships one prebuilt)

illustrative numbers (simulated)



a human-only reference hides the mouse cells — and conserved reads leak into human homolog counts

Count each barcode's human vs mouse UMIs: on-axis = one species, stuck in the middle = doublet or contamination — the barnyard plot

Think downstream: B16's CNV analysis is waiting

Gene names come from the GTF

the names in your matrix are decided by the GTF count used today

CNV needs gene coordinates

inferCNV takes a gene-position file; mismatched versions silently drop genes

One version, whole project

tumor projects compare across patients — never switch versions mid-project

Episode checklist

- ✓ you can state what R1 (barcode+UMI), R2 (cDNA), I1/I2 (index) hold
- ✓ read lengths checked with zcat: 28 bp = v3/v4, 26 bp = v2
- ✓ filenames match SampleName_S1_L001_R1_001.fastq.gz
- ✓ human/mouse, no transgene → prebuilt 10x refdata, version recorded
- ✓ custom builds: matching FASTA/GTF releases, mkgtf before mkref
- ✓ transgenes added to both FASTA and GTF, GFP counts verified nonzero

All six boxes ticked before count gets to run in B3

ONE-LINE SUMMARY

FASTQ answers who, what, and which sample; the reference decides which genes can be counted.

With both in hand and verified, count is just one command.

Check yourself 1

In 10x 3' data, which file actually gets aligned to the genome?

A R1

B R2

C I1

D Both R1 and R2

Answer B. Only R2 is cDNA. R1 is just 28 bp of cell barcode plus UMI — identity, not content; I1/I2 are sample indices whose job ends at demultiplexing. Completely different from bulk R1/R2.

Check yourself 2

You have mouse data and must also count vector-borne mCherry. What about the reference?

A Use the 10x prebuilt mm10/GRCm39 as is

B Rebuild from scratch with fresh Ensembl files

C Extend the prebuilt: add mCherry to both FASTA and GTF, then mkref

D No change — mCherry is detected automatically

Answer C. Mouse means the prebuilt is your base, but mCherry is not in it: append to the FASTA, add a GTF entry, rerun mkref. One-sided edits fail — FASTA alone is trap 3, counting zero forever.

Check yourself 3

Same FASTQ, different reference, and gene counts differ by 2,000+. Most likely cause?

A Unstable sequencing quality

B Different GTF annotation version and filter list

C Randomness across cellranger versions

D Different cell numbers

Answer B. The reads did not change, so only the reference can. The FASTA barely moves across years; the GTF does — newer annotations add genes (lncRNA especially) and filter whitelists add or drop whole classes. Hence: always report the reference version.

NEXT EPISODE

B3: Running count and reading web_summary

FASTQ ready, reference ready — one command, a dozen output files.

Which do you open first? Which numbers say the data is usable? Next: web_summary, panel by panel.